Can you send me everything in detail about the library noise and seat detection project

Below is a complete, course-ready plan for the **Smart Library: IoT-Based Seat Availability and Noise Monitoring System**, including the mmWave enhancement we discussed. The recommended design is realistic for an Arduino Mega, demonstrable in a small model, and can stay near the HKD 150 budget.

Smart Library Project

1. Project overview

Suggested title

> **Smart Library: A Privacy-Preserving Multi-Sensor Seat Availability and Noise Monitoring System Using mmWave Presence Radar**

One-paragraph proposal

This project develops an Arduino Mega-based Smart Library prototype that monitors study-seat availability and relative noise conditions in a quiet study zone. Individual seat sensors detect whether each seat is free or occupied, while a 24 GHz mmWave radar sensor provides privacy-preserving human-presence verification, including detection of stationary occupants. A microphone-based sensor classifies ambient sound activity as quiet, moderate, or noisy using calibrated thresholds and duration filtering. The system displays a live seat map, available-seat count, noise state, and zone-presence status on an OLED display. RGB LEDs and a buzzer provide gentle feedback when sustained excessive noise occurs. The prototype will be evaluated through seat-detection accuracy, mmWave false-positive rate, response time, noise-classification performance, and sensor-disagreement analysis.

Problem addressed

Students often enter a library or study room without knowing:

- Whether seats are available.
- Which specific seats are free.
- Whether an apparently available space is actually occupied by a quiet/stationary student.
- Whether a quiet zone is currently suitable for focused study.
- Whether high noise is a brief event or persistent disruption.

The project gives immediate feedback:

```
SMART LIBRARY
Quiet Zone A

Free seats: 2 / 4
Noise: QUIET
Zone presence: DETECTED
```

The design is particularly appropriate for quiet study areas because many university libraries distinguish silent, quiet, and collaborative zones with different expected behaviour. Brown University, for example, uses a multi-level system that includes “Absolute Quiet” at 0–25 dB, “Considerate” at 26–40 dB, and “Conversational” at 41–60 dB. Your low-cost prototype should not claim official calibrated dB measurement, but the policy concept supports quiet/moderate/noisy feedback.[1] [2]

System design

2. Main functions

Function	Description	Output
Individual seat detection	Detects whether each monitored seat is free or occupied	Per-seat state
Available-seat count	Calculates total free seats	OLED count
Seat map	Shows the status of every monitored seat	OLED symbols / seat LEDs
mmWave zone detection	Detects human presence, including a still student	Zone-active status
Noise monitoring	Measures relative ambient sound activity	Quiet/moderate/noisy state
Sustained-noise alert	Warns only if noise remains excessive for a chosen period	OLED + RGB LED + optional buzzer
Sensor-fusion checking	Detects disagreement between seat map and zone presence	Diagnostic warning
Zone settings	Supports silent, quiet, or collaborative modes	Threshold selection
Event logging, optional	Stores occupancy/noise events	Serial Monitor / microSD
Wireless dashboard, optional	Sends data to phone/laptop	Bluetooth module

3. Recommended scope

Build a **four-seat study zone model**, not an entire library:

```
┌──────────────────────────────────────────────┐
│              SMART LIBRARY: ZONE A           │
│                 Quiet Study Area             │
│                                              │
```

```
|    [ Seat 1 ]              [ Seat 2 ]        |
|     IR/FSR                   IR/FSR           |
|                                               |
|    [ Seat 3 ]              [ Seat 4 ]        |
|     IR/FSR                   IR/FSR           |
|                                               |
|  mmWave presence sensor → monitors zone       |
|  Sound sensor → monitors ambient activity     |
|                                               |
|  OLED: 2 / 4 seats free, QUIET                |
|  RGB LED: green/yellow/red                    |
└───────────────────────────────────────────────┘
```

Why four seats?

- Four sensors are enough to show a meaningful seat map.
- Four seats fit physically on a cardboard or foam-board model.
- It leaves pins and budget for the display, microphone, mmWave radar, LEDs, and buzzer.
- It is easier to test thoroughly than six or more sensors.
- You can show scale-up potential in your report.

Hardware choice

4. Recommended components

Component	Quantity	Purpose	Essential?
Arduino Mega 2560	1	Main controller	Yes
IR obstacle sensors **or** FSR sensors	4	Individual seat occupancy	Yes
Low-cost microphone/sound sensor	1	Relative noise monitoring	Yes
OLED display	1	Dashboard and seat map	Yes
RGB LED	1	Overall noise/system status	Yes
Buzzer	1	Sustained-noise warning	Recommended
24 GHz mmWave radar, e.g. LD2410/LD2410C	1	Zone presence / stationary-person verification	Strong enhancement
Seat LEDs + resistors	4	Each seat's local free/occupied status	Optional but recommended
Push button	1–3	Change zone mode / acknowledge warning	Recommended
Cardboard / foam board / miniature chairs	—	Physical library model	Yes
HC-05 Bluetooth module	1	Wireless dashboard	Optional
microSD module	1	Event logging	Optional

The Arduino Mega is suitable because it has 54 digital I/O pins, 16 analog inputs, four hardware UARTs, and support for SPI and I^2C. It therefore has enough interfaces for four seat sensors, a microphone, display, LEDs, buzzer, and UART-connected mmWave radar. [3]

5. Seat sensor recommendation

Best low-cost option: IR obstacle sensors

Mount one IR obstacle sensor at each seat in the model.

Possible placement:

- Under the chair, aimed upward.
- Under the desk edge, aimed toward the chair area.
- Above a model chair, aimed downward.
- In front of the seat, aimed at the expected body/object location.

```
Side view:

      Desk
  ┌─────────────┐
  │             │
  └─────────────┘
        ↓
    IR sensor
        ↓
    [ Seat ]
```

Rule:

```
Detected object near target seat → Occupied
No object detected → Free
```

Alternative: FSR pressure sensor

Place an FSR under the seat surface.

Advantages:

- More direct “someone is sitting” evidence.
- Not sensitive to room lighting.
- Better conceptually for a real chair.

Disadvantages:

- May cost more.
- Must be mechanically supported.

- Sensor readings vary with object placement and weight.

Avoid PIR as the only seat sensor

A PIR sensor detects changes in infrared radiation caused by movement. A student reading quietly may remain nearly still, so the sensor can eventually report no motion even though the seat is occupied.

Avoid several ultrasonic sensors close together

Ultrasonic sensors can work, but their echoes can interfere if several trigger simultaneously. They require sequential triggering and more physical space. For the four-seat version, IR or FSR sensors are simpler.

mmWave enhancement

6. Role of mmWave radar

The mmWave sensor should **not** be the only seat detector. Use it as:

1. A **quiet-zone presence verifier**, or
2. An "advanced seat" sensor for one selected seat.

The recommended use is **zone presence verification**:

```
IR / FSR seat sensors → individual seat map
mmWave radar → someone is present in the zone
```

This is useful because mmWave radar can detect stationary human presence and micro-motion, unlike a PIR sensor. Arduino documents an `ld2410` library for the Hi-Link LD2410 24 GHz FMCW radar; its range-dependent sensitivity can be configured for static and moving targets. [4]

Why mmWave is a good choice

Feature	mmWave advantage
Privacy	It does not capture visual camera images
Still person detection	Can detect a person sitting quietly
Contactless operation	No wearable device or pressure pad required
Zone coverage	One sensor can monitor a small study zone
Technical value	Adds UART communication, calibration, and sensor fusion
Modern relevance	Used in smart building occupancy applications

The LD2410 family is a 24 GHz FMCW human-presence radar designed for stationary and moving targets. It typically communicates through UART and can provide a simple digital

output; its sensitivity across distance ranges can be configured.[5] [4]

7. Important mmWave limitation

A low-cost mmWave presence module may detect:

- A student in an adjacent seat.
- Someone walking through a nearby aisle.
- Someone behind a thin divider.
- Reflections from nearby objects.
- Presence outside the intended seat boundary.

Therefore, do **not** claim:

> "One mmWave module identifies every occupied seat."

Instead, claim:

> "mmWave provides privacy-preserving zone-level human presence verification and supports sensor fusion with seat-level sensors."

Sensor-fusion rule

$$
\begin{bmatrix}
\text{ZoneOccupied} \\
\text{mmWavePresence} \\
\lor \\
\left(\sum_{i=1}^{4} \text{SeatOccupied}_i > 0 \right)
\end{bmatrix}
$$

You can detect disagreement:

Seat sensors	mmWave	System interpretation
All seats free	No presence	Zone likely empty
One/more seats occupied	Presence	Normal agreement
All seats free	Presence	Possible untracked person, passing person, or false positive
Seats occupied	No presence	Possible mmWave blind spot or sensor calibration issue

Dashboard example:

```
SYSTEM CHECK
Seat map: 0 occupied
mmWave: Presence detected
```

```
Possible untracked activity.
```

This is a valuable feature for an engineering project because you are not hiding sensor limitations—you are detecting and reporting them.

8. mmWave module connection

For an LD2410-like module:

mmWave pin	Arduino Mega connection	Purpose
VCC	5 V, subject to module specification	Power
GND	GND	Common ground
TX	Mega RX1, pin 19	Radar sends serial data to Mega
RX	Mega TX1, pin 18, through level conversion if needed	Mega sends configuration to radar
OUT	D22, optional	Simple HIGH/LOW presence status

Many LD2410 modules use:

- 5 V supply.
- 3.3 V UART logic.
- Default UART rate of 256000 baud.
- A simple `OUT` pin that goes HIGH when presence is detected. [6] [7] [5]

Very important voltage note

The Arduino Mega transmits 5 V logic from TX1, but many mmWave modules expect 3.3 V on their RX input.

Use one of these:

- A logic-level converter.
- A resistor voltage divider between Mega TX1 and mmWave RX.
- Only use the `OUT` pin if you do not need serial configuration.

The sensor TX output at 3.3 V is usually high enough for the Mega RX1 input to read as a logical HIGH, but confirm your exact module's datasheet.

Simple `OUT` mode

This is easiest:

```
const int mmwaveOutPin = 22;

void setup() {
  pinMode(mmwaveOutPin, INPUT);
```

```
}

void loop() {
  bool presenceDetected = digitalRead(mmwaveOutPin);

  if (presenceDetected) {
    // Zone active
  } else {
    // Zone empty
  }
}
```

UART mode

Use UART mode if you want richer data such as:

- Moving target detection.
- Stationary target detection.
- Estimated target distance.
- Distance-gate energy.
- Sensitivity/range configuration.

Because LD2410 uses 256000 baud, use a **hardware UART** such as `Serial1` on the Mega, not `SoftwareSerial`. The Mega's additional hardware serial interfaces are one reason it is a good fit. [8] [9]

Noise monitoring

9. Noise sensor design

Use a low-cost microphone module, such as a module with:

- Analog output `AO`.
- Optional digital threshold output `DO`.
- Onboard potentiometer for rough sensitivity adjustment.

Use the **analog output** for better classification. Connect it to an analog pin, for example `A0`.

```
Microphone sensor AO → Arduino Mega A0
Microphone sensor VCC → 5 V
Microphone sensor GND → GND
```

10. What the sensor measures

A cheap microphone module is not a calibrated sound-pressure-level meter. Its Arduino reading does not automatically equal decibels.

Correct terms:

- Relative ambient sound level.
- Noise activity.
- Quiet / moderate / high sound condition.
- Noise threshold exceeded.

Avoid saying:

- “This room is exactly 45 dB.”
- “The system uses legally accurate decibel measurement.”

Your project is still valid: the aim is to identify whether the sound activity is appropriate for the configured library zone.

11. Noise-processing algorithm

Sample the microphone for a short window, such as 100 ms. Find the minimum and maximum ADC readings:

$$A = x_{\max} - x_{\min}$$

Where:

- $x_{\max}$ is the maximum microphone value in the window.
- $x_{\min}$ is the minimum microphone value in the same window.
- A is the peak-to-peak sound activity amplitude.

Then smooth the amplitude:

$$
[
\overline{A}[k]
$$

\frac{1}{M}
\sum_{i=0}^{M-1} A[k-i]
]

Where M might be 5–10 measurement windows.

Example classification

```
Average amplitude < 30:
    QUIET

30 to 79:
    MODERATE

80 or above:
    NOISY
```

These are **not universal values.** You must calibrate them with your actual microphone, room, sensor placement, and power supply.

12. Sustained-noise filtering

A clap, chair scrape, dropped pen, or door closing should not create a major quiet-zone alert.

Use duration filtering:

$$
[
\text{NoiseWarning}
$$

(\overline{A} > T_{high})
\land
(t_{high} > T_{warning})
]

For example:

Duration of high sound	System reaction
Less than 3 seconds	Ignore / continue monitoring
3–5 seconds	Show “Noise rising” / yellow LED
More than 5 seconds	Display quiet-zone reminder / red LED
More than 15 seconds	Log the event and give one short beep

This makes the project more realistic and avoids annoying users.

13. Noise modes

Add a button so your system can select one of three zone policies:

Mode	Intended use	Threshold
Silent	Deep individual study	Lowest noise threshold
Quiet	Normal quiet study	Medium threshold
Collaborative	Group study/discussion	Highest threshold

Mathematically:

$$T_{\text{silent}} < T_{\text{quiet}} < T_{\text{collaborative}}$$

This adds configurability and lets you explain that libraries do not all use one noise standard. Quiet and collaborative spaces usually have different behavioural expectations. [10] [11] [12]

User interface

14. OLED dashboard layout

Default screen

```
SMART LIBRARY
Zone A: Quiet Study

Seats free: 2 / 4
Noise: QUIET
Presence: YES
```

Seat-map screen

```
SEAT MAP

[1 X]  [2 O]
[3 X]  [4 O]

O = free
X = occupied
```

Noise warning screen

```
QUIET ZONE NOTICE

Noise is high.
Please lower your voice.
```

Full-zone screen

```
ZONE A FULL

Free seats: 0 / 4
Please check another zone.
```

Sensor-disagreement screen

```
SENSOR CHECK

Seat map: all free
mmWave: presence
Check zone / calibration
```

15. LED and buzzer behavior

Condition	RGB LED	Buzzer	OLED
Zone normal and quiet	Green	Off	Normal dashboard
Moderate noise	Yellow	Off	“Moderate noise”
Sustained high noise	Red	One short beep, optional	Quiet-zone message
No seats available	Blue or purple	Off	“Zone full”
Sensor disagreement	Flashing yellow	Off	Diagnostic message
Hardware/sensor error	Flashing red	Short error pattern	Error message

Avoid continuous buzzing. In a library setting, the alert device should not become another source of noise.

Software design

16. System state machine

```
INITIALIZE
  ├── Set up pins, display, thresholds
  ├── Check sensor availability
  └── MONITOR

MONITOR
  ├── Read seat sensors → UPDATE_SEATS
  ├── Read mmWave state → UPDATE_PRESENCE
  ├── Read microphone → CHECK_NOISE
  ├── Button press → SETTINGS
  └── Repeat

UPDATE_SEATS
  ├── Apply seat debounce/filter
  ├── Recalculate occupied/free count
  ├── Update OLED/seat LEDs
  └── Return to MONITOR

UPDATE_PRESENCE
  ├── Compare mmWave vs seat map
  ├── If disagreement → DIAGNOSTIC_STATE
  └── Return to MONITOR
```

```
CHECK_NOISE
  ├── Quiet/moderate → MONITOR
  ├── High noise, short duration → WATCH_NOISE
  └── High noise, sustained → NOISE_WARNING

WATCH_NOISE
  ├── Sound becomes acceptable → MONITOR
  └── Duration exceeds threshold → NOISE_WARNING

NOISE_WARNING
  ├── Red LED + OLED reminder
  ├── Optional one-shot beep
  ├── Log event
  └── Return to MONITOR when sound decreases

SETTINGS
  ├── Change zone mode
  ├── Change thresholds
  ├── Recalibrate seat/noise baselines
  └── Return to MONITOR
```

17. Seat debounce logic

Do not change a seat from free to occupied after one sensor reading. Use stable detection over time.

Example:

```
Seat becomes occupied:
Sensor reports occupied continuously for 3 seconds.

Seat becomes free:
Sensor reports free continuously for 10 seconds.
```

This prevents a bag briefly passing near the sensor from immediately changing the seat map.

Pseudocode:

```
if (rawSeatState != confirmedSeatState) {
    if (millis() - stateChangeStartTime[i] > confirmationTime) {
        confirmedSeatState = rawSeatState;
    }
} else {
    stateChangeStartTime[i] = millis();
}
```

18. Main program structure

```
void loop() {
  readSeatSensors();
  updateSeatStates();

  readMmwavePresence();
  checkSensorAgreement();

  readNoiseLevel();
  updateNoiseState();

  readButtons();
  updateDisplay();
  updateLEDsAndBuzzer();

  logEventsIfNeeded();
}
```

Avoid long `delay()` calls. Use `millis()` for timing so that seat monitoring, noise monitoring, display updates, and button presses all continue working.

Pin plan

19. Example Arduino Mega wiring allocation

Device	Arduino Mega connection
IR seat sensor 1	D22
IR seat sensor 2	D23
IR seat sensor 3	D24
IR seat sensor 4	D25
mmWave OUT	D26
mmWave TX	RX1, pin 19
mmWave RX	TX1, pin 18 through level shifting
Sound sensor analog output	A0
OLED SDA	SDA, pin 20
OLED SCL	SCL, pin 21
RGB LED red	D2 through resistor
RGB LED green	D3 through resistor
RGB LED blue	D4 through resistor
Buzzer	D5
Seat LED 1	D27 through resistor

Device	Arduino Mega connection
Seat LED 2	D28 through resistor
Seat LED 3	D29 through resistor
Seat LED 4	D30 through resistor
Zone-mode button	D31 using `INPUT_PULLUP`
Acknowledge button	D32 using `INPUT_PULLUP`
Optional HC-05 Bluetooth	Serial2, pins 16/17
Optional SD card	SPI pins 50–53 plus a separate CS pin

The Mega has hardware UART, I²C, and SPI capability suitable for this arrangement.[3]

Build plan

20. Construction plan

Stage 1: Seat model

Build a four-seat library model from cardboard or foam board:



Place a small object, card, book, or model person in each seat to simulate occupation.

Stage 2: Individual seat sensing

- Wire one IR or FSR sensor.
- Write code that reports `FREE` or `OCCUPIED`.
- Calibrate threshold and placement.
- Repeat for all four seats.
- Add one LED per seat.

Stage 3: OLED seat map

- Add OLED.
- Display each seat state.
- Calculate free-seat count.

$$
[
N_{free}
$$

4

$$
\sum_{i=1}^{4}
SeatOccupied_i
]
$$

Stage 4: Noise sensing

- Add microphone analog output to `A0`.
- Print raw values to Serial Monitor.
- Measure quiet baseline.
- Measure normal talking.
- Measure loud or sustained talking.
- Choose initial thresholds.
- Add duration filtering.

Stage 5: RGB/buzzer feedback

- Green for quiet.
- Yellow for moderate.
- Red for sustained high noise.
- One short beep only for sustained noise.

Stage 6: Add mmWave radar

- First connect only `OUT` to confirm simple presence detection.
- Position the radar above a small zone or one advanced seat.
- Test stationary-person detection.
- Check adjacent-seat and walk-by false triggers.
- If stable, add UART data/configuration.

Stage 7: Sensor-fusion feature

Display:

```
Seats: 0 / 4 occupied
mmWave: presence

Possible untracked person
```

Then adjust radar placement/range, sensor placement, or state thresholds.

Stage 8: Enclosure and presentation

- Label each seat.
- Label scan/sensor locations.
- Mount OLED visibly.
- Hide wires where possible.
- Add project title and explanation card.
- Use a power bank or safe USB supply for the final demo.

Testing and evaluation

21. Test plan

Test category	Test procedure	Metric
Seat accuracy	Test each seat empty and occupied 20–30 times	Accuracy
Seat response	Add/remove object from each seat	Mean response time
Seat false positive	Leave seats empty for 10–20 minutes	False-positive count
Noise classification	Test silence, quiet speech, normal conversation, clapping	Correct classification rate
Noise false alert	Run normal quiet activity	False-warning count
Sustained-noise response	Make elevated noise for known durations	Alert trigger-time error
mmWave stationary presence	Person stays still for 1, 5, 10 minutes	Detection success rate
mmWave walk-by test	Person walks past zone while empty	False-trigger rate
Adjacent-seat test	Occupy nearby/non-target seat	Cross-detection rate
Fusion disagreement	Compare seat map and radar result in controlled situations	Disagreement rate
Usability	Ask classmates to read dashboard state	Correct interpretation percentage

22. Key equations

Seat detection accuracy

$$
\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \times 100\%
$$

Where:

- TP: Occupied seat correctly reported as occupied.
- TN: Empty seat correctly reported as free.
- FP: Empty seat incorrectly reported as occupied.
- FN: Occupied seat incorrectly reported as free.

False-positive rate

$$
\text{False Positive Rate} = \frac{FP}{FP + TN} \times 100\%
$$

Seat response time

$$
\overline{t}_{response} = \frac{1}{N} \sum_{i=1}^{N} \left(t_{system\ update,i} - t_{actual\ state\ change,i} \right)
$$

Noise-alert timing error

$$
\left|
t_{\text{actual alert}}
-
t_{\text{expected alert}}
\right|
$$

mmWave presence-detection rate

$$
\text{Presence Detection Rate}
=
\frac{\text{Correct presence detections}}
{\text{Total presence trials}}
\times 100\%
$$

Budget

23. Estimated HKD 150 budget

Assuming the Mega, OLED, RGB LED, buzzer, buttons, and basic wiring are already supplied by your lab board:

Item	Quantity	Approximate cost
IR obstacle sensors	4	HKD 20–50
Analog microphone module	1	HKD 10–20
LD2410/LD2410C-style mmWave module	1	HKD 35–70
LEDs + resistors	4–6	HKD 10–20
Logic-level converter or resistor-divider parts	1 set	HKD 5–15
Cardboard / foam board / model materials	—	HKD 15–35
Jumper wires / spare components	—	HKD 10–20
Estimated total		**HKD 105–230**

Because prices vary widely, the full mmWave version may exceed HKD 150. Use this purchase priority:

1. **Four seat sensors.**
2. **Sound sensor.**
3. **Model materials and LEDs.**
4. **mmWave module only if the remaining budget allows.**
5. Bluetooth/SD logging only if your team already has the parts.

Lower-cost version

If mmWave is too expensive, the project remains strong with:

- Four IR or FSR seat sensors.
- Microphone module.
- OLED seat map.
- RGB/buzzer.
- Good calibration and testing.

Then describe mmWave as a **future system upgrade** in your report.

Alternative scope with one mmWave sensor

If you can afford one mmWave module but must simplify elsewhere:

- Two seat sensors rather than four.
- One mmWave advanced seat.
- One microphone.
- OLED + RGB LED.
- Compare IR versus mmWave performance.

This creates a research-style project:

> **Comparative Evaluation of IR and mmWave Sensors for Privacy-Preserving Study-Seat Occupancy Detection**

Final feature set

24. Minimum viable system

- Four monitored seats.
- One digital sensor per seat.
- OLED free-seat count.
- Basic seat map.
- One analog microphone sensor.
- Quiet/moderate/noisy classification.

- RGB LED feedback.
- Cardboard model.

25. Strong final system

- Four individual seat sensors with filtering.
- OLED seat map and free-seat count.
- Analog noise sampling and moving-average filtering.
- Duration-based noise warnings.
- Silent/quiet/collaborative zone modes.
- RGB LED and one-shot buzzer alerts.
- One mmWave zone-presence sensor.
- Sensor disagreement reporting.
- Quantitative test results.

26. Optional extensions

- Bluetooth dashboard using HC-05.
- microSD event logging.
- Two zones: Quiet Zone and Collaborative Zone.
- Doorway people counter using two IR beams.
- RFID student check-in / seat reservation.
- Temperature, humidity, and lighting comfort monitoring.
- A web dashboard if you later use ESP32 instead of Mega.

Important limitations

Be explicit about these in your proposal and final report:

- Low-cost IR and FSR sensors require calibration and can misclassify unusual object placement.
- A low-cost microphone module gives relative sound activity, not calibrated decibel values.
- mmWave radar provides presence detection but may not accurately identify a specific seat among several adjacent seats.
- mmWave performance depends heavily on mounting, range configuration, furniture, and nearby movement.
- The system monitors a small model or study zone; it is not a production-scale library system.
- The buzzer must be brief and limited so it does not itself disturb library users.

- Presence sensing should not be used to identify individuals; the project intentionally avoids cameras for privacy.

Final recommendation

Build the project in this order:

1. Four IR seat sensors and seat map.
2. OLED dashboard and individual-seat LEDs.
3. Microphone noise classification with persistence filtering.
4. RGB LED and quiet-zone alert.
5. One mmWave module as a zone-presence verifier.
6. Sensor-fusion disagreement display.
7. Collect test data and complete the physical model.

That approach gives you a complete working project even if the mmWave module takes longer to configure. The final version will demonstrate multi-sensor embedded design, privacy-aware occupancy sensing, real-time user feedback, and evaluation based on accuracy and false alarms.

⁂

1. https://library.brown.edu/create/libnews/sound-levels/
2. https://www.browndailyherald.com/article/2024/03/university-library-announces-new-sound-level-system
3. https://www.wevolver.com/specs/arduino-mega-2560-rev3
4. https://docs.arduino.cc/libraries/ld2410/
5. https://ld2410.com/
6. https://www.tinkered.ai/components/ld2410c-mmwave-radar
7. https://seengreat.com/upload/file/86/HLK+LD2410+Life+Presence+Sensor+Module+Manual+V1.03(220629).pdf
8. https://voltdocs.dev/components/sensor.motion.ld2410/
9. https://github.com/mgiesen/ld2410
10. https://www.ohio.edu/library/about/policies-guidelines/study-space-noise-guidelines
11. https://www.reading.ac.uk/library/about-us/policies-rules/noise
12. https://library.csun.edu/about/policies/noise
13. https://www.lib.ncku.edu.tw/journal/pdf/22/22_04.pdf
14. https://www.espboards.dev/sensors/ld2410/
15. https://m.indiamart.com/proddetail/autobotix-hlk-ld2410-human-presence-radar-sensor-module-2858885937155.html
16. https://docs.arduino.cc/resources/datasheets/A000067-datasheet.pdf
17. https://docs.arduino.cc/hardware/mega-2560/
18. https://store-usa.arduino.cc/products/arduino-mega-2560-rev3

19. https://library.brown.edu/create/libnews/page/17/?query-30-page=33